

Camera Calibration and Photogrammetry

Computer Vision: AI3619

Camera Calibration and Photogrammetry

Method to find a camera's parameters and a method to estimate 3D structure using two cameras.

Topics:

- (1) Linear Camera Model
- (2) Camera Calibration
- (3) Photogrammetry and Stereo

Forward Imaging Model: 3D to 2D

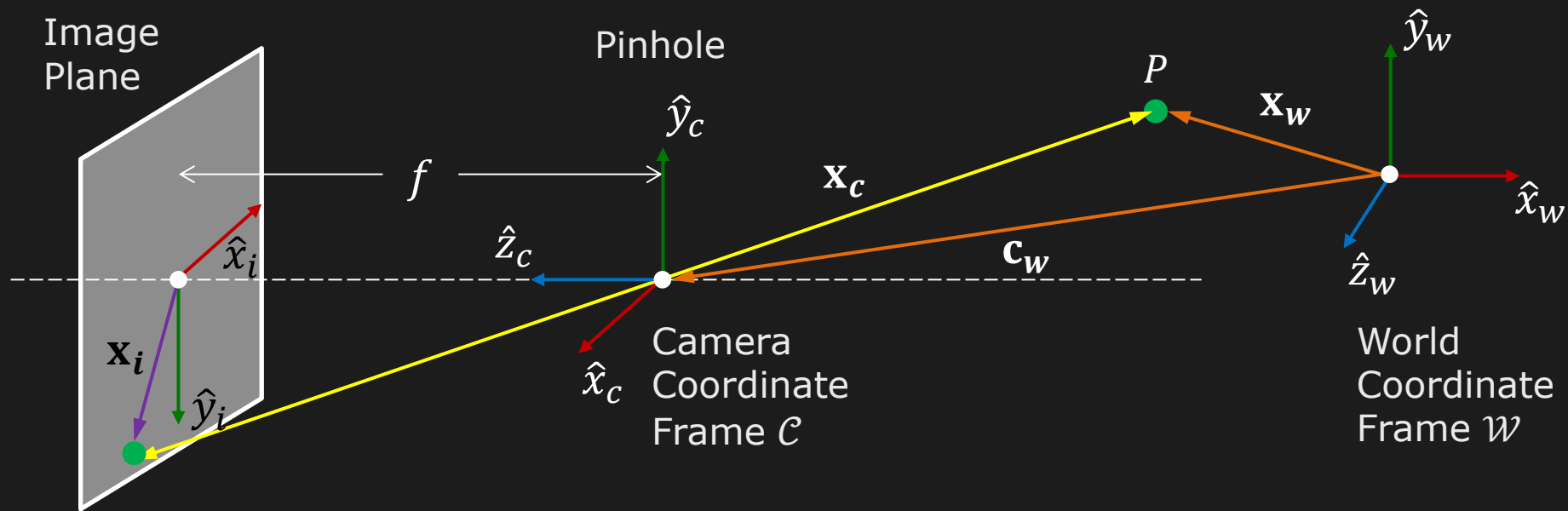


Image Plane
Coordinates

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

Camera
Coordinates

$$\mathbf{x}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

World
Coordinates

$$\mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$$

Perspective
Projection

Coordinate
Transformation

Forward Imaging Model: 3D to 2D

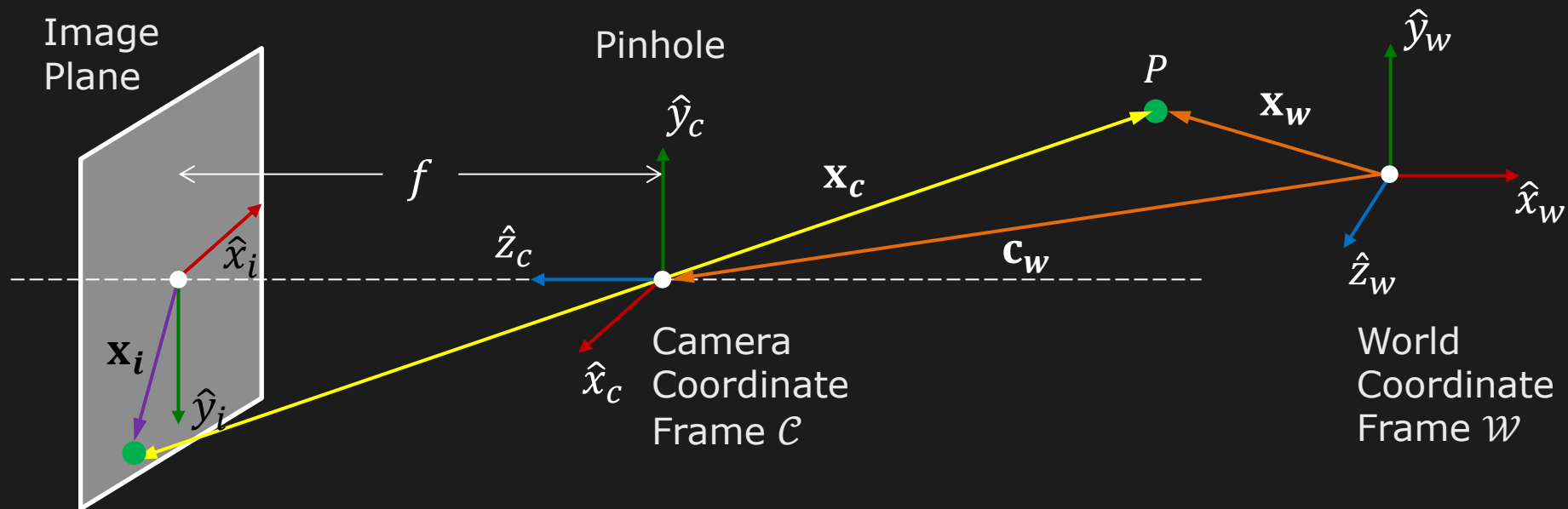


Image Plane
Coordinates

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

Camera
Coordinates

$$\mathbf{x}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

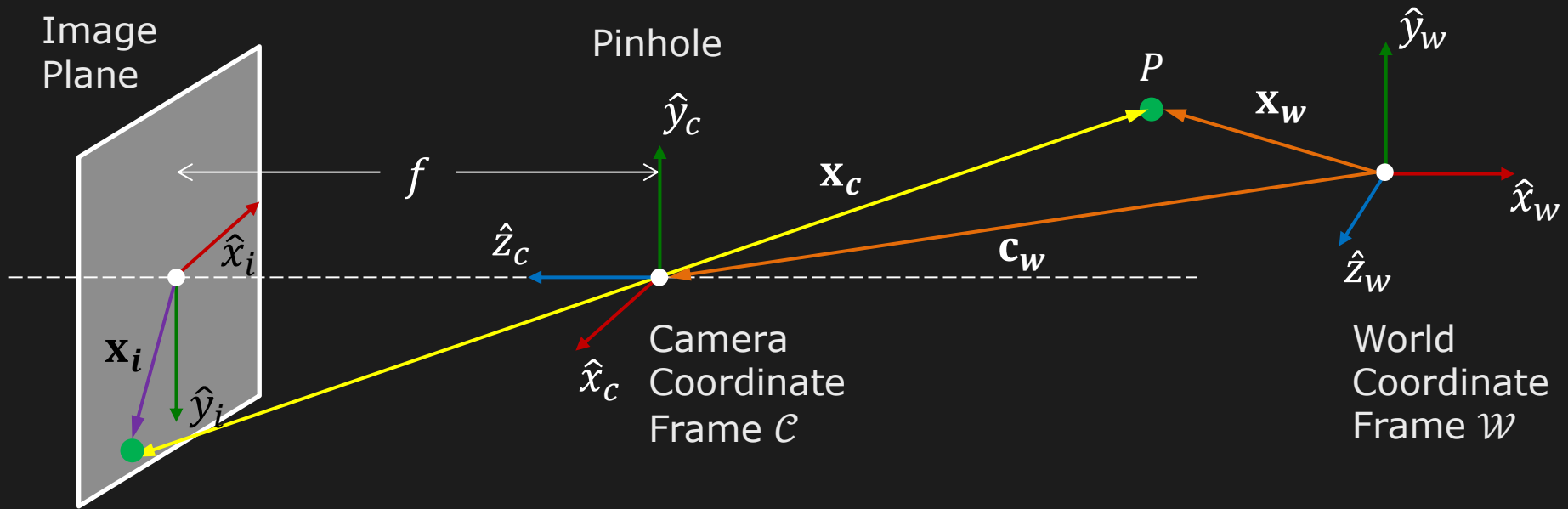
World
Coordinates

$$\mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$$

Perspective
Projection

Coordinate
Transformation

Perspective Projection

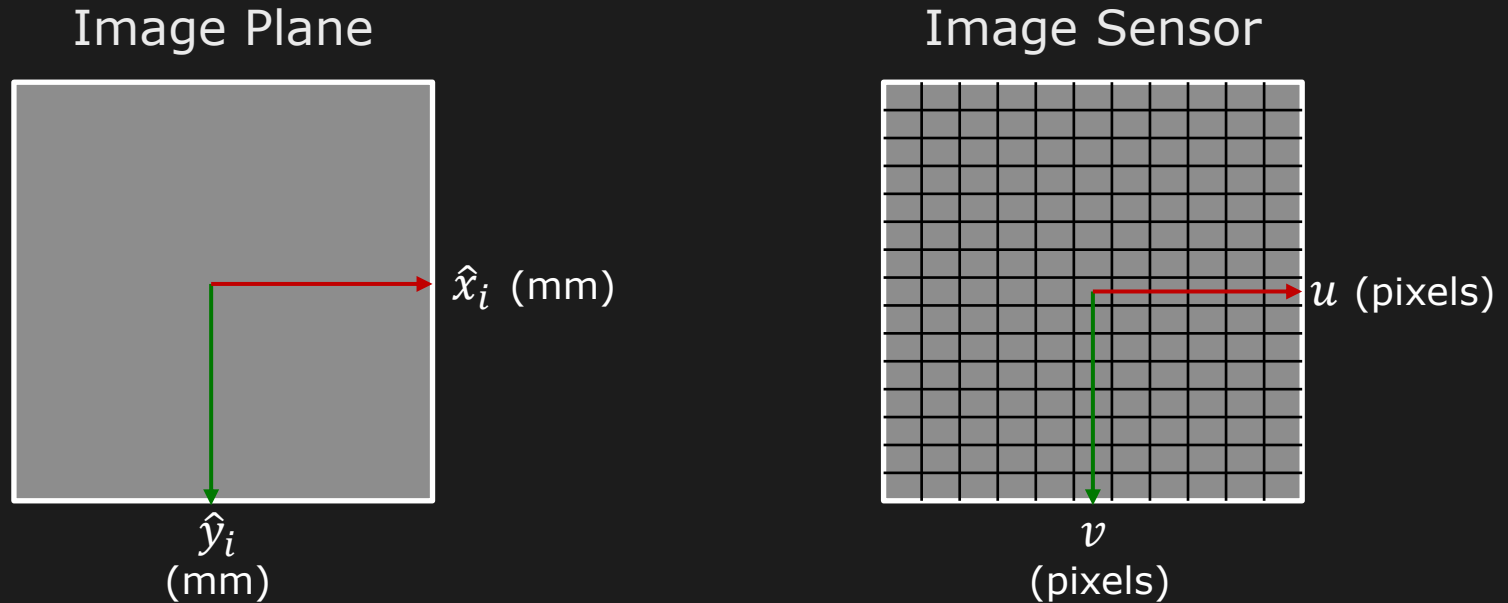


We know that: $\frac{x_i}{f} = \frac{x_c}{z_c}$ and $\frac{y_i}{f} = \frac{y_c}{z_c}$

Therefore: $x_i = f \frac{x_c}{z_c}$ and $y_i = f \frac{y_c}{z_c}$

What are the units of x_i ?

Image Plane to Image Sensor Mapping



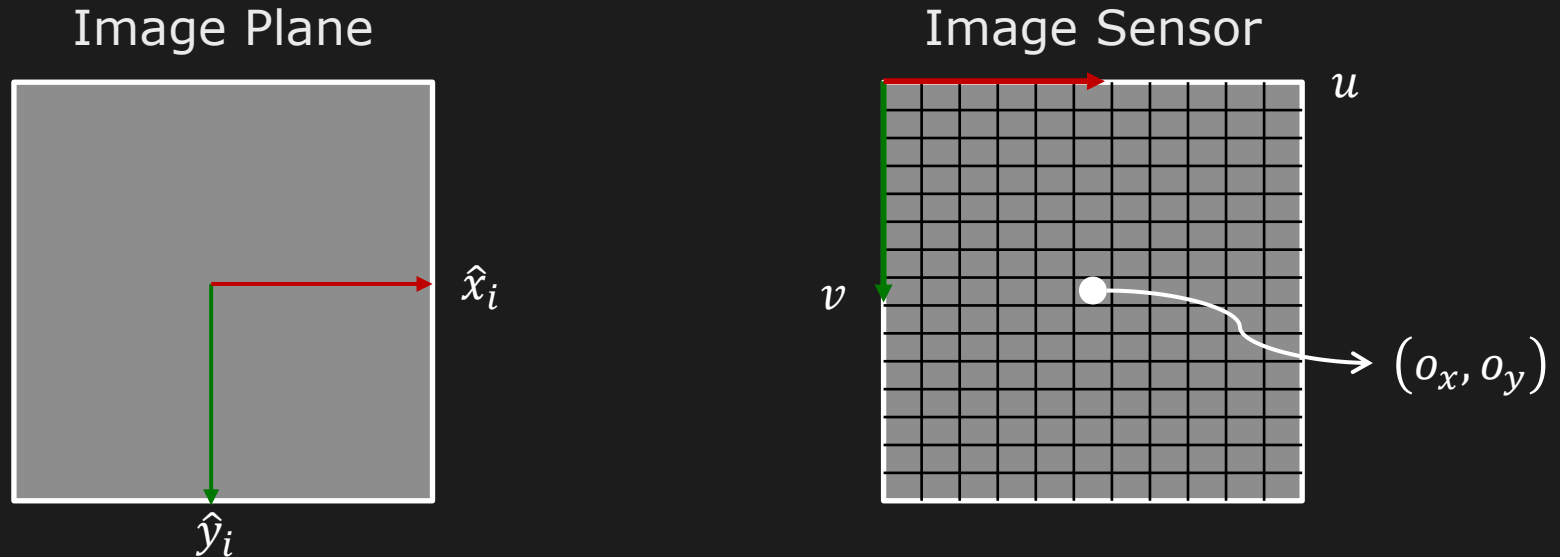
Pixels may be rectangular.

If m_x and m_y are the pixel densities (ex: pixels/mm) in x and y directions respectively, then pixel coordinates are:

$$u = m_x x_i = m_x f \frac{x_c}{z_c}$$

$$v = m_y y_i = m_y f \frac{y_c}{z_c}$$

Image Plane to Image Sensor Mapping



We usually treat the top-left corner of the image sensor as its origin (easier for indexing). If the optical axis passes through (o_x, o_y) (**Principle Point**) on the sensor, then:

$$u = m_x f \frac{x_c}{z_c} + o_x$$

$$v = m_y f \frac{y_c}{z_c} + o_y$$

Perspective Projection

$$u = m_x f \frac{x_c}{z_c} + o_x$$

$$v = m_y f \frac{y_c}{z_c} + o_y$$

$$u = f_x \frac{x_c}{z_c} + o_x$$

$$v = f_y \frac{y_c}{z_c} + o_y$$

where: $(f_x, f_y) = (m_x f, m_y f)$ are the focal lengths in pixels in x and y directions, respectively.

(f_x, f_y, o_x, o_y) : Intrinsic parameters of the camera.
They represent the camera's internal geometry.

Perspective Projection

$$u = m_x f \frac{x_c}{z_c} + o_x$$

$$v = m_y f \frac{y_c}{z_c} + o_y$$

$$u = f_x \frac{x_c}{z_c} + o_x$$

$$v = f_y \frac{y_c}{z_c} + o_y$$

Equations for Perspective projection are **Non-Linear**.

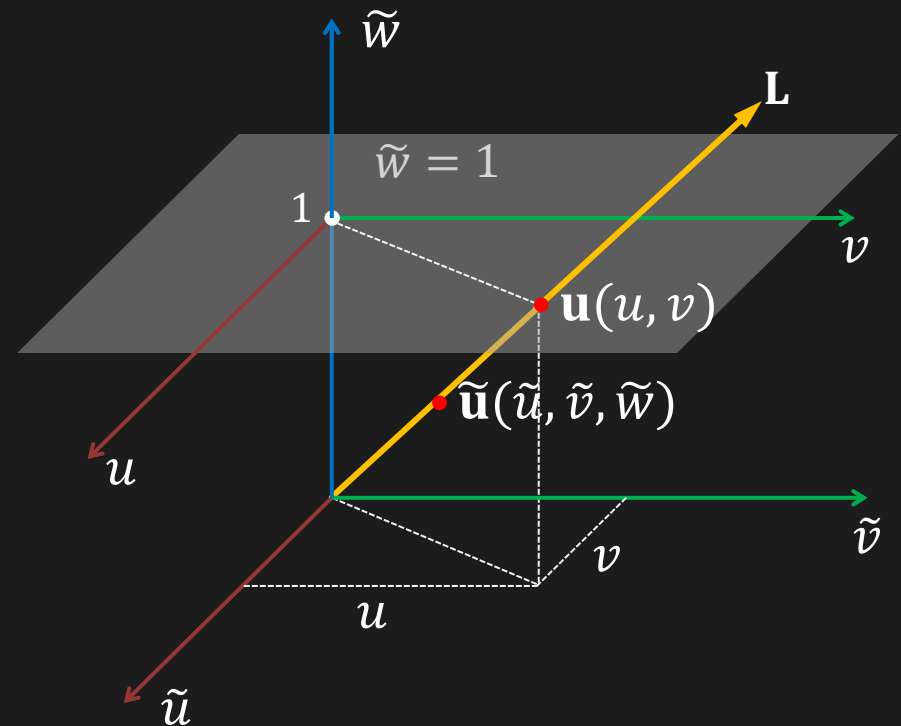
It is often convenient to express them as linear equations.

Homogenous Coordinates

The **homogenous** representation of a 2D point $\mathbf{u} = (u, v)$ is a 3D point $\tilde{\mathbf{u}} = (\tilde{u}, \tilde{v}, \tilde{w})$. The third coordinate $\tilde{w} \neq 0$ is fictitious such that:

$$u = \frac{\tilde{u}}{\tilde{w}} \quad v = \frac{\tilde{v}}{\tilde{w}}$$

$$\mathbf{u} \equiv \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{w}u \\ \tilde{w}v \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \tilde{\mathbf{u}}$$



Every point on line L (except origin) represents the homogenous coordinate of $\mathbf{u}(u, v)$

Homogenous Coordinates

The **homogenous** representation of a 3D point $\mathbf{x} = (x, y, z)$ is a 4D point $\tilde{\mathbf{x}} = (\tilde{x}, \tilde{y}, \tilde{z}, \tilde{w})$. The fourth coordinate $\tilde{w} \neq 0$ is fictitious such that:

$$x = \frac{\tilde{x}}{\tilde{w}} \quad y = \frac{\tilde{y}}{\tilde{w}} \quad z = \frac{\tilde{z}}{\tilde{w}}$$

$$\mathbf{x} \equiv \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{w}x \\ \tilde{w}y \\ \tilde{w}z \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \\ \tilde{w} \end{bmatrix} = \tilde{\mathbf{x}}$$

Perspective Projection in Homogenous Coordinates

Perspective projection equations:

$$u = f_x \frac{x_c}{z_c} + o_x \qquad v = f_y \frac{y_c}{z_c} + o_y$$

Homogenous coordinates of (u, v) :

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} z_c u \\ z_c v \\ z_c \end{bmatrix} = \begin{bmatrix} f_x x_c + z_c o_x \\ f_y y_c + z_c o_y \\ z_c \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

where: $(u, v) = (\tilde{u}/\tilde{w}, \tilde{v}/\tilde{w})$

Intrinsic Matrix

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

Calibration Matrix:

$$K = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

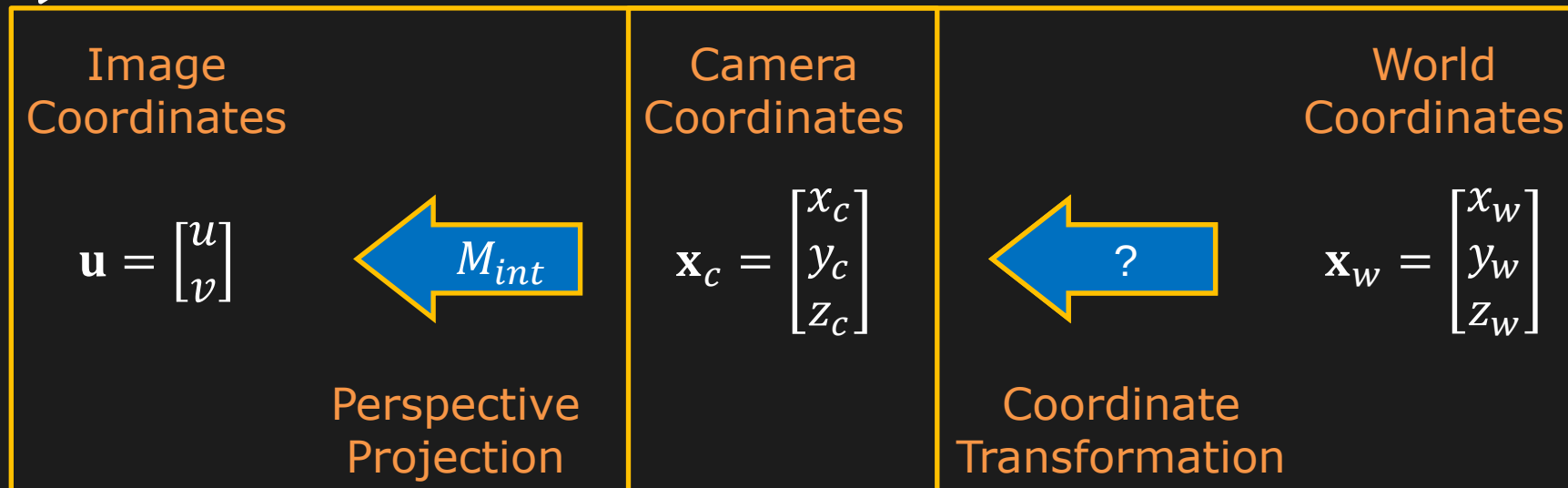
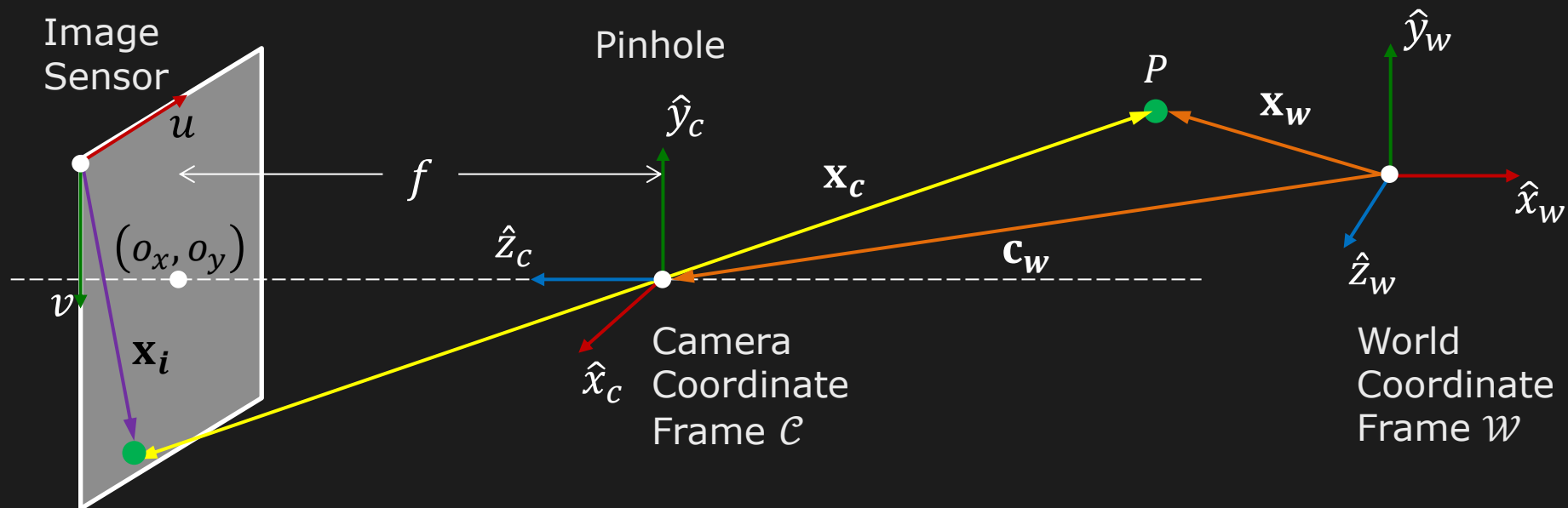
Intrinsic Matrix:

$$M_{int} = [K|0] = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

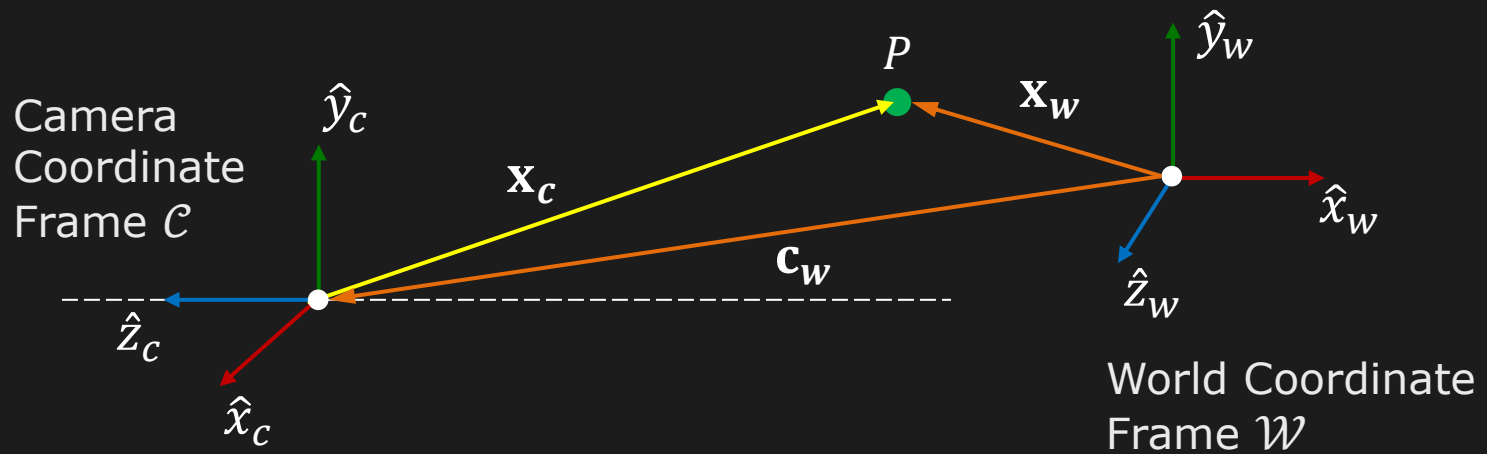
Upper Right Triangle Matrix

$$\tilde{\mathbf{u}} = [K|0] \tilde{\mathbf{x}}_c = M_{int} \tilde{\mathbf{x}}_c$$

Forward Imaging Model: 3D to 2D

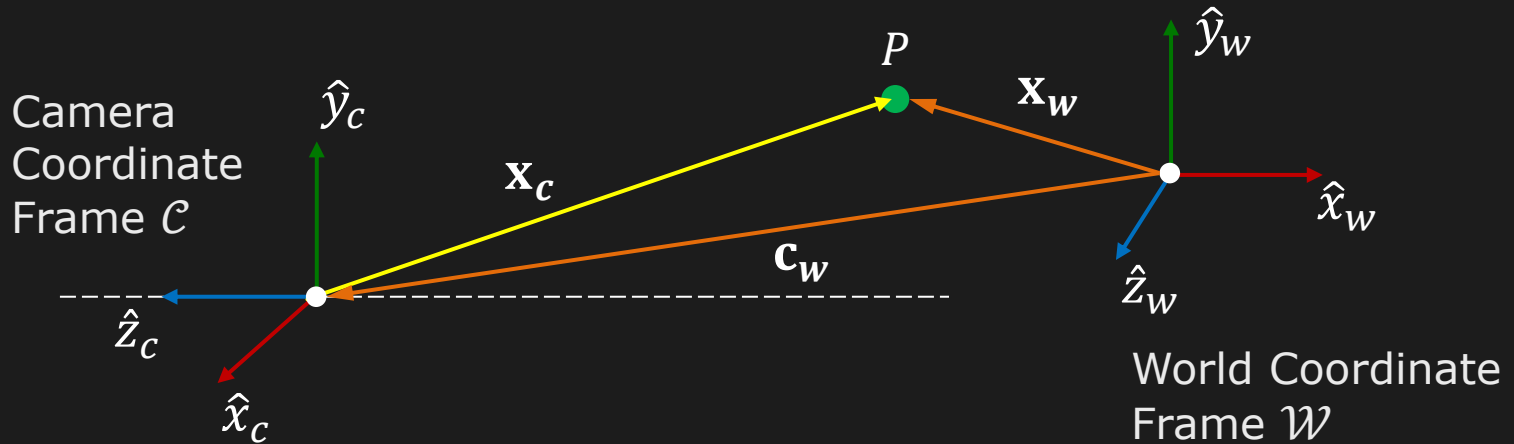


Extrinsic Parameters



Position $\mathbf{c}_w$ and **Orientation** R of the camera in the world coordinate frame $\mathcal{W}$ are the camera's **Extrinsic Parameters**.

World-to-Camera Transformation



Given the **extrinsic parameters** ($R, \mathbf{c}_w$) of the camera, the camera-centric location of any point $\mathbf{x}_w$ in the world coordinate frame is:

$$\mathbf{x}_c = R(\mathbf{x}_w - \mathbf{c}_w) = R\mathbf{x}_w - R\mathbf{c}_w = R\mathbf{x}_w + \mathbf{t} \quad (\mathbf{t} = -R\mathbf{c}_w)$$

$$\mathbf{x}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}}_{\text{Rotation}} \underbrace{\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}}_{\text{World Point}} + \underbrace{\begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}}_{\text{Translation}}$$

Rotation

Translation

World-to-Camera Transformation

Rewriting using homogenous coordinates:

$$\tilde{\mathbf{x}}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Extrinsic Matrix: $M_{ext} = \begin{bmatrix} R_{3 \times 3} & \mathbf{t}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$

$$\tilde{\mathbf{x}}_c = M_{ext} \tilde{\mathbf{x}}_w$$

Forward Imaging Model: 3D to 2D

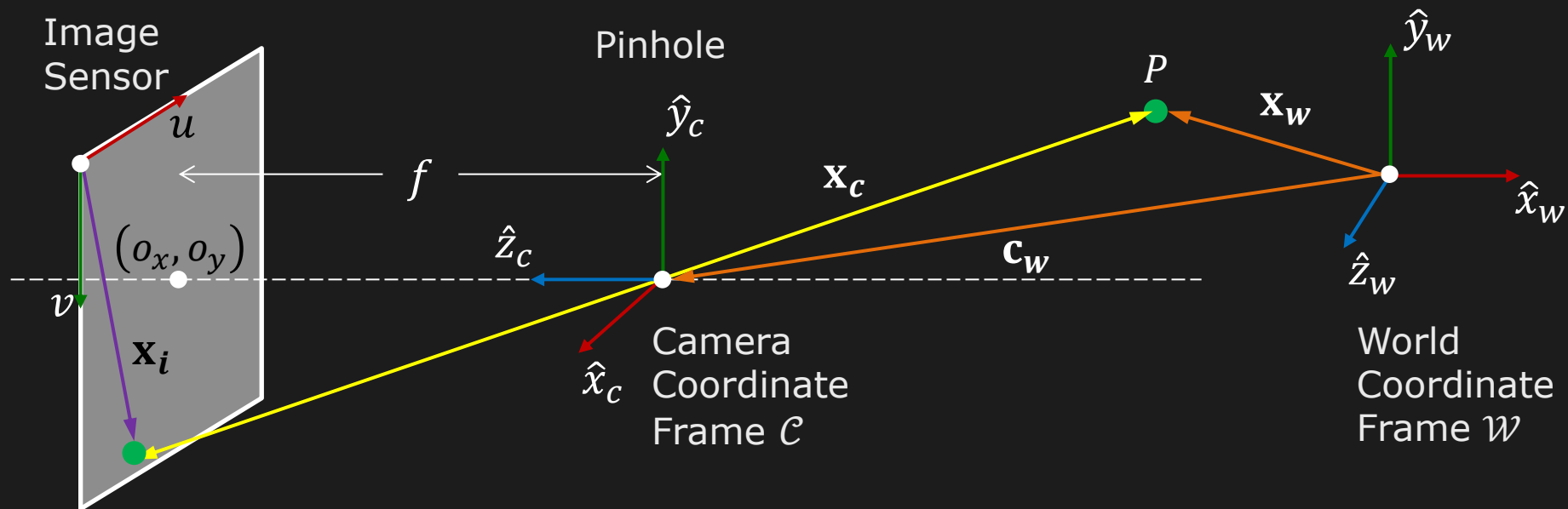


Image
Coordinates

$$\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix}$$

Perspective
Projection



Camera
Coordinates

$$\mathbf{x}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

Coordinate
Transformation



World
Coordinates

$$\mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$$

Linear Camera Model

Camera to Pixel

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} f_x & s & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{u}} = M_{int} \tilde{\mathbf{x}}_c$$

World to Camera

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{x}}_c = M_{ext} \tilde{\mathbf{x}}_w$$

Combining the above two equations, we get the
Projection Matrix P :

$$\tilde{\mathbf{u}} = M_{int} M_{ext} \tilde{\mathbf{x}}_w = \mathbf{P} \tilde{\mathbf{x}}_w$$

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Scale of Projection Matrix

Projection matrix acts on homogenous coordinates.

We know that:

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \equiv k \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \quad (k \neq 0 \text{ is any constant})$$

That is:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} \equiv k \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Therefore, Projection Matrix P and kP produce the same homogenous pixel coordinates.

Projection Matrix P needs to be determined only up to a scale factor.

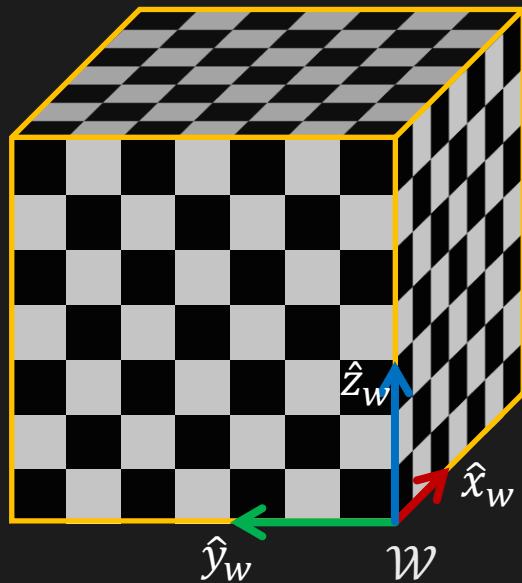
Camera Calibration

Most vision applications require the knowledge of intrinsic (f_x, f_y, o_x, o_y) and extrinsic $(R, \mathbf{t})$ parameters of the cameras being used.

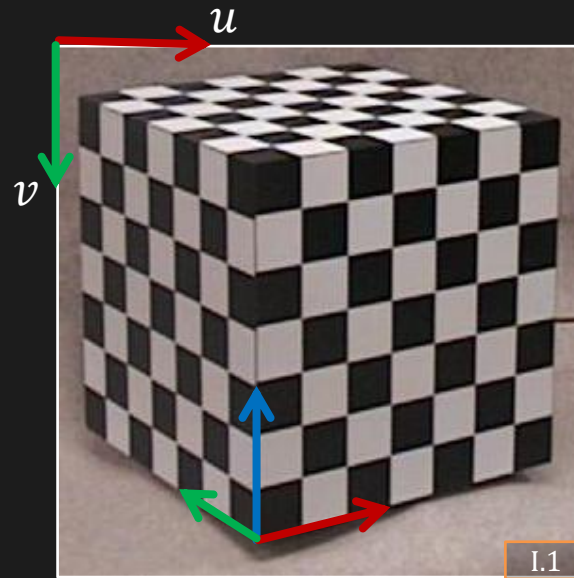
We “Calibrate” the cameras to determine these.

Camera Calibration Procedure

Step 1: Capture an image of an object with known geometry.



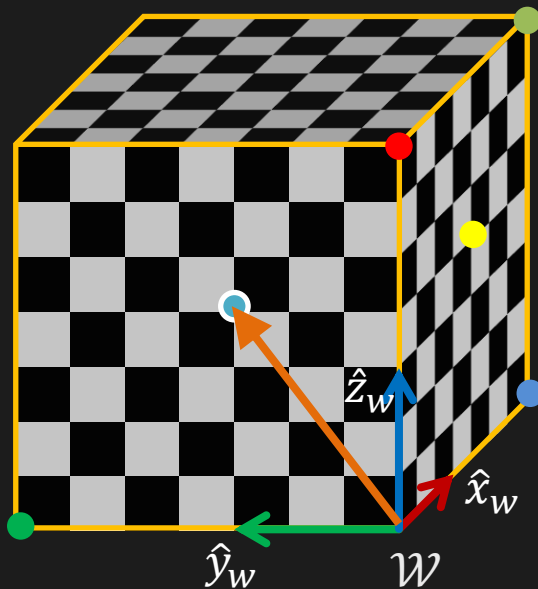
Object whose precise geometry is known



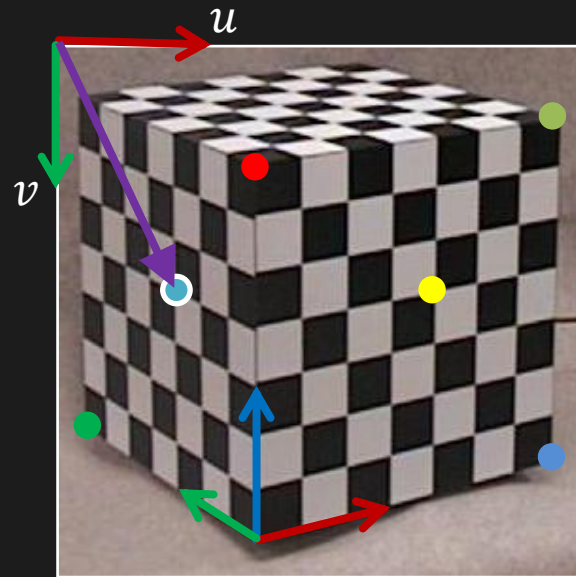
Captured Image

Camera Calibration Procedure

Step 2: Identify correspondence between 3D scene points and captured image.



Object whose precise geometry is known



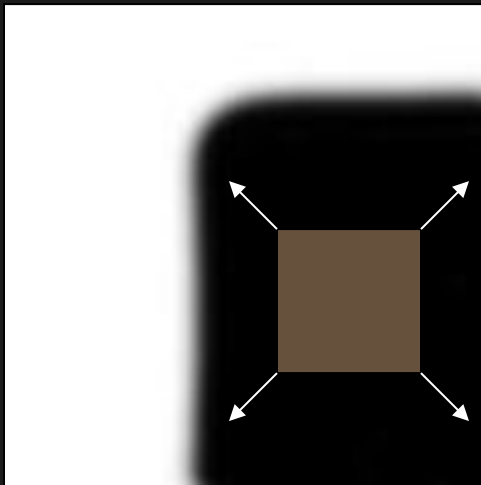
Captured Image

$$\bullet \mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}$$

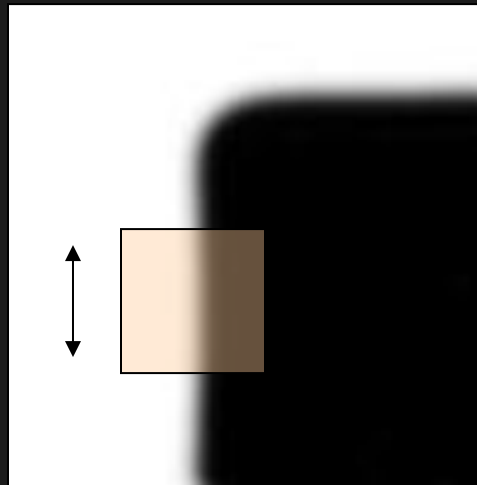
$$\bullet \mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 56 \\ 115 \end{bmatrix}$$

Corner Detection: Basic Idea

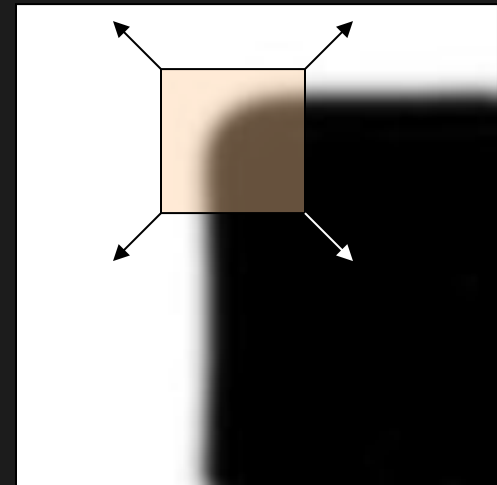
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give a *large change* in intensity



“flat” region:
no change in
all directions



“edge”:
no change
along the edge
direction



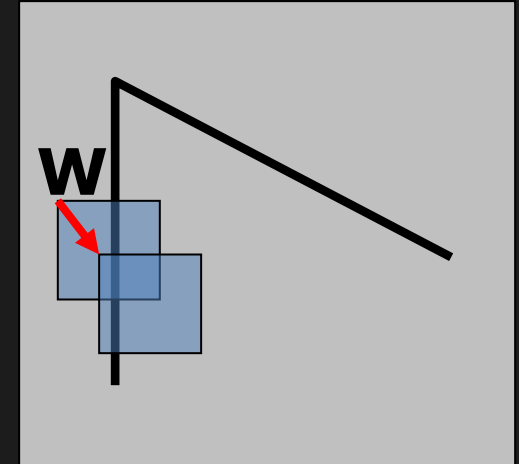
“corner”:
significant
change in all
directions

Corner Detection: The Math

Consider shifting the window **W** by (u, v)

- How do the pixels in **W** change?
- Compare each pixel before and after by summing up the squared differences (SSD)
- This defines an SSD “error” of $E(u, v)$:

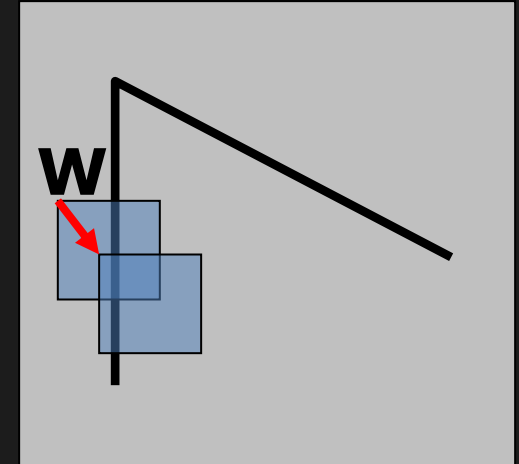
$$E(u, v) = \sum_{(x, y) \in W} [I(x + u, y + v) - I(x, y)]^2$$



Corner Detection: The Math

Consider shifting the window **W** by (u, v)

- How do the pixels in **W** change?
- Compare each pixel before and after by summing up the squared differences (SSD)
- This defines an SSD “error” of $E(u, v)$:

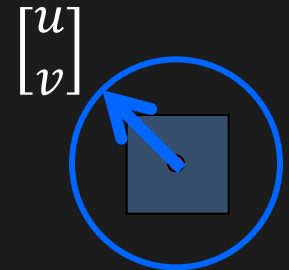
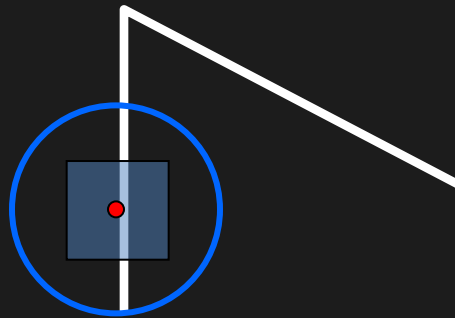


$$\begin{aligned} E(u, v) &= \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2 \\ &\approx \sum_{(x,y) \in W} \left[I(x, y) + [I_x \quad I_y] \begin{bmatrix} u \\ v \end{bmatrix} - I(x, y) \right]^2 \\ &\approx \sum_{(x,y) \in W} \left[[I_x \quad I_y] \begin{bmatrix} u \\ v \end{bmatrix} \right]^2 \end{aligned}$$

Corner Detection: The Math

This can be rewritten:

$$E(u, v) = [u \quad v] \underbrace{\sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} u \\ v \end{bmatrix}$$



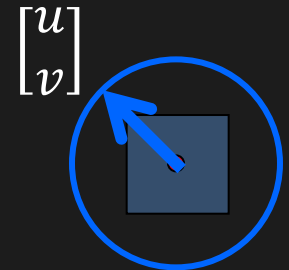
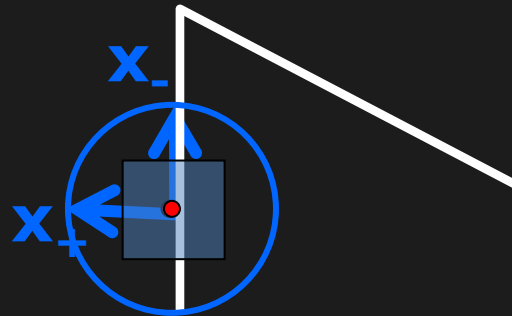
For the example above

- You can move the center of the blue window to anywhere on the blue unit circle
- Which directions will result in the largest and smallest E values?
- We can find these directions by looking at the eigenvectors of $\mathbf{H}$

Corner Detection: The Math

This can be rewritten:

$$E(u, v) = [u \quad v] \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$



Eigenvalues and eigenvectors of H

- Define shifts with the smallest and largest change (E value)
- x_+ = direction of **largest** increase in E.
- λ_+ = amount of increase in direction x_+
- x_- = direction of **smallest** increase in E.
- λ_- = amount of increase in direction x_+

$$Hx_+ = \lambda_+ x_+$$

$$Hx_- = \lambda_- x_-$$

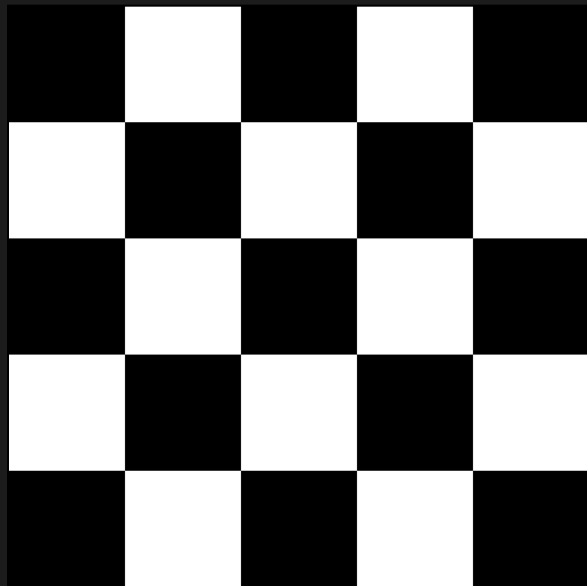
Corner Detection: The Math

How are λ_+ , $\mathbf{x}_+$, λ_- , and $\mathbf{x}_-$ relevant for corner detection?

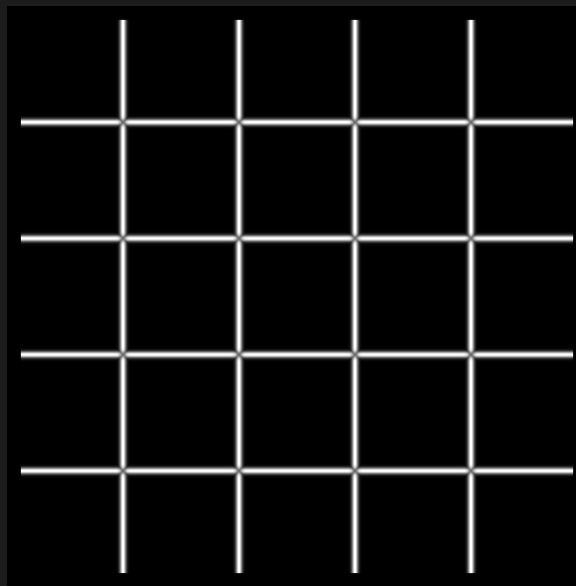
- What's our feature scoring function?

Want $E(u,v)$ to be **large** for small shifts in **all** directions

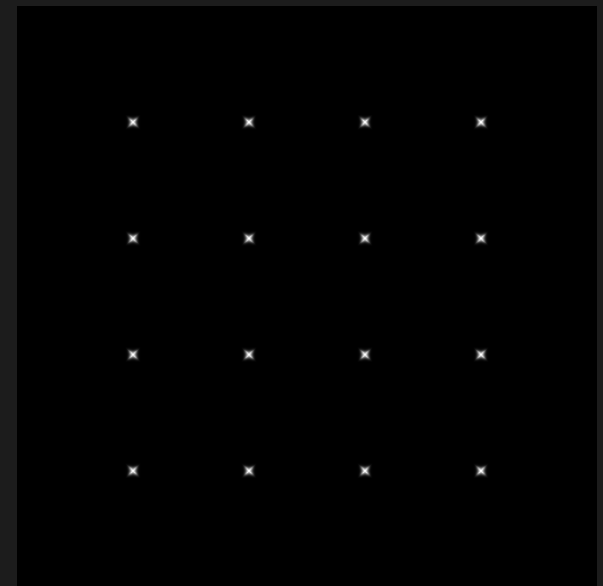
- The *minimum* of $E(u,v)$ should be large, over all unit vectors $[u \ v]$
- This minimum is given by the smaller eigenvalue (λ_-) of $\mathbf{H}$



$\mathbf{I}$



λ_+

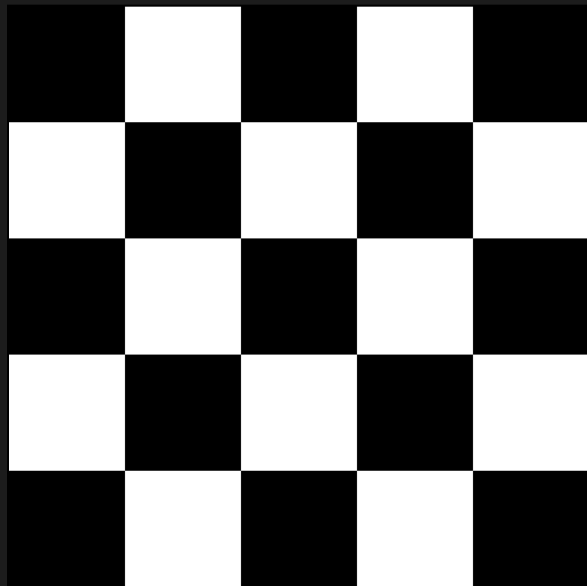


λ_-

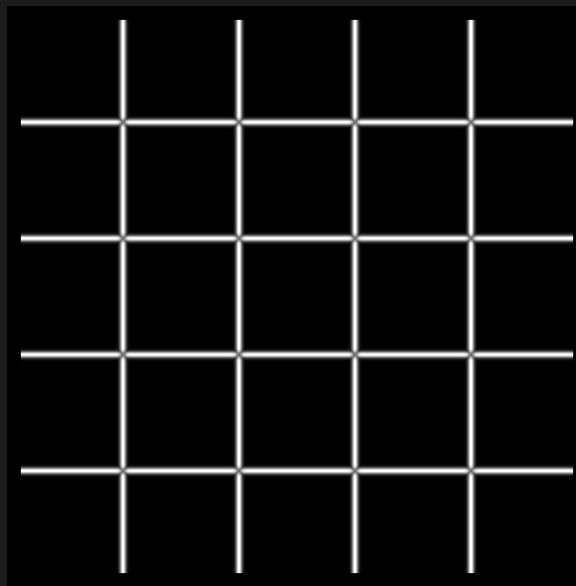
Corner Detection: The Math

Here's what you do

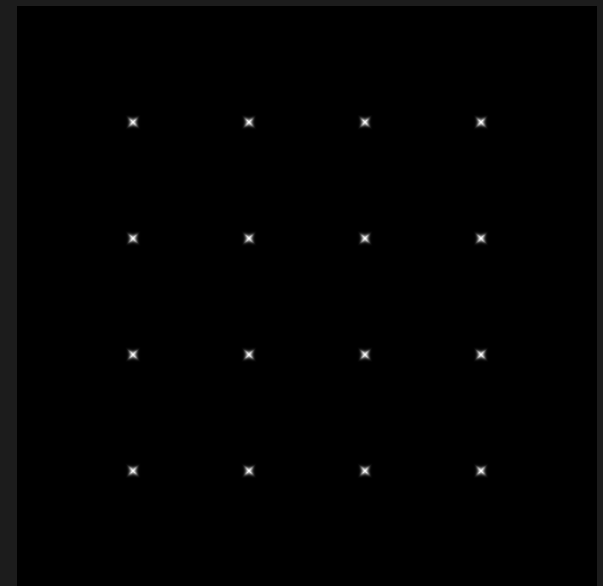
- Compute the gradient at each point in the image
- Create the ***H*** matrix from the entries in the gradient
- Compute the eigenvalues.
- Find points with large response ($\lambda_- > \text{threshold}$)
- Choose those points where λ_- is a local maximum as corners



I



λ_+



λ_-

Camera Calibration Procedure

Step 3: For each corresponding point i in scene and image:

$$\underbrace{\begin{bmatrix} u^{(i)} \\ v^{(i)} \\ 1 \end{bmatrix}}_{\text{Known}} \equiv \underbrace{\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}}_{\text{Unknown}} \underbrace{\begin{bmatrix} x_w^{(i)} \\ y_w^{(i)} \\ z_w^{(i)} \\ 1 \end{bmatrix}}_{\text{Known}}$$

Expanding the matrix as linear equations:

$$u^{(i)} = \frac{p_{11}x_w^{(i)} + p_{12}y_w^{(i)} + p_{13}z_w^{(i)} + p_{14}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

$$v^{(i)} = \frac{p_{21}x_w^{(i)} + p_{22}y_w^{(i)} + p_{23}z_w^{(i)} + p_{24}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

Camera Calibration Procedure

Step 4: Rearranging the terms:

$$\begin{bmatrix}
 x_w^{(1)} & y_w^{(1)} & z_w^{(1)} & 1 & 0 & 0 & 0 & 0 & -u_1 x_w^{(1)} & -u_1 y_w^{(1)} & -u_1 z_w^{(1)} & -u_1 \\
 0 & 0 & 0 & 0 & x_w^{(1)} & y_w^{(1)} & z_w^{(1)} & 1 & -v_1 x_w^{(1)} & -v_1 y_w^{(1)} & -v_1 z_w^{(1)} & -v_1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 x_w^{(i)} & y_w^{(i)} & z_w^{(i)} & 1 & 0 & 0 & 0 & 0 & -u_i x_w^{(i)} & -u_i y_w^{(i)} & -u_i z_w^{(i)} & -u_i \\
 0 & 0 & 0 & 0 & x_w^{(i)} & y_w^{(i)} & z_w^{(i)} & 1 & -v_i x_w^{(i)} & -v_i y_w^{(i)} & -v_i z_w^{(i)} & -v_i \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 x_w^{(n)} & y_w^{(n)} & z_w^{(n)} & 1 & 0 & 0 & 0 & 0 & -u_n x_w^{(n)} & -u_n y_w^{(n)} & -u_n z_w^{(n)} & -u_n \\
 0 & 0 & 0 & 0 & x_w^{(n)} & y_w^{(n)} & z_w^{(n)} & 1 & -v_n x_w^{(n)} & -v_n y_w^{(n)} & -v_n z_w^{(n)} & -v_n
 \end{bmatrix}
 \begin{bmatrix}
 p_{11} \\
 p_{12} \\
 p_{13} \\
 p_{14} \\
 p_{21} \\
 p_{22} \\
 p_{23} \\
 p_{24} \\
 p_{31} \\
 p_{32} \\
 p_{33} \\
 p_{34}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0
 \end{bmatrix}$$

A
 (Known)

p
 (Unknown)

Step 5: Solve for p :

$$A p = 0$$

Least Squares Solution for P

$$A \mathbf{p} = \mathbf{0}$$

If $\bar{\mathbf{p}}$ is a solution, so is $k\bar{\mathbf{p}}$ for any constant k .

But, Projection Matrix P needs to be determined only up to a scale factor. We can assume any scale for $\mathbf{p}$.

Set scale so that: $\|\mathbf{p}\|^2 = 1$

We want $A\mathbf{p}$ as close to 0 as possible and $\|\mathbf{p}\|^2 = 1$:

$$\min_{\mathbf{p}} \|A\mathbf{p}\|^2 \text{ such that } \|\mathbf{p}\|^2 = 1$$

Solution: eigenvector $\mathbf{p}$ with **smallest eigenvalue** λ of matrix $A^T A$

Rearrange solution $\mathbf{p}$ to form the projection matrix P .

Extracting Intrinsic and Rotation Parameters

We know that:

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \begin{matrix} \text{(Intrinsic)} \\ \begin{bmatrix} f_x & s & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix} \begin{matrix} \text{(Extrinsic)} \\ \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$

That is:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = KR$$

Given that K is an **Upper Right Triangle** matrix and R is an **Orthonormal** matrix, it is possible to “**decouple**” K and R from their product using **RQ factorization**.

(See Appendix)

Extracting Translation Parameters

We know that:

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \begin{matrix} \text{(Intrinsic)} \\ \begin{bmatrix} f_x & s & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix} \begin{matrix} \text{(Extrinsic)} \\ \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$

That is:

$$\begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix} = \begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} = K\mathbf{t} = -KR\mathbf{c}_w \quad (\mathbf{t} = -R\mathbf{c}_w)$$

Therefore:

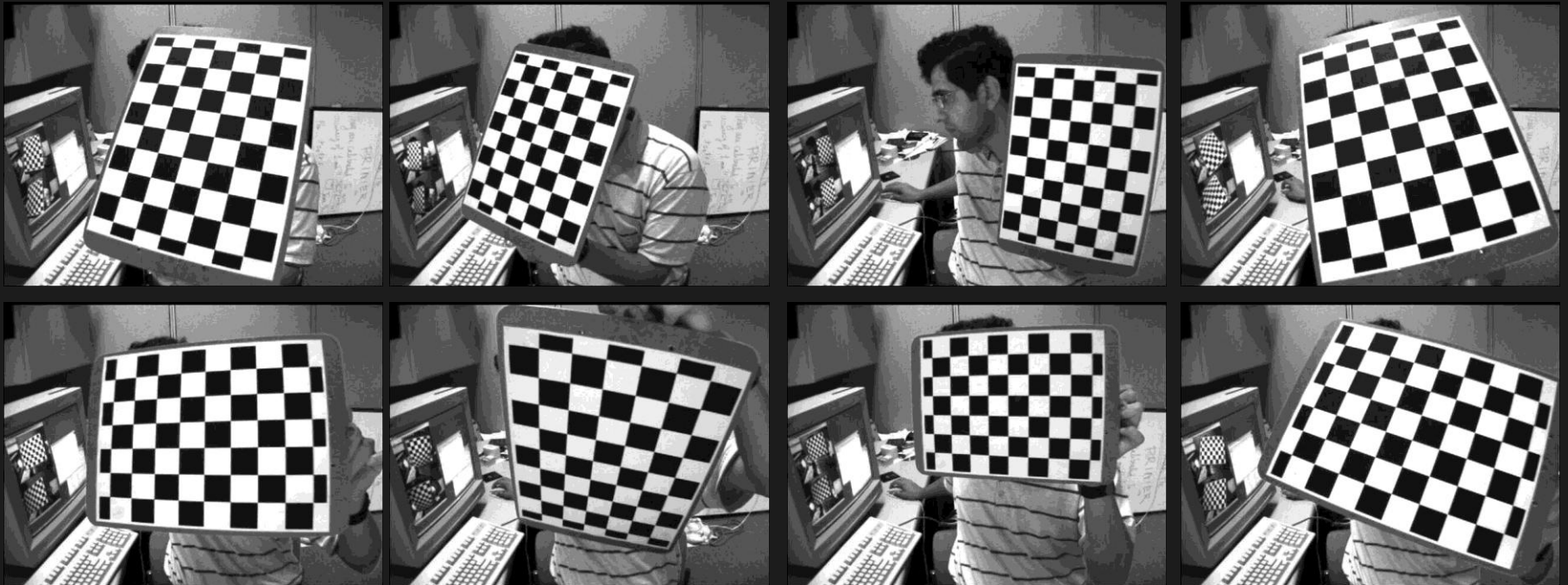
$$\mathbf{t} = K^{-1} \begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix}$$

$$\mathbf{c}_w = -R^T K^{-1} \begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix}$$

Camera Calibration

So what's the procedure to calibrate a camera?

First, we generally collect "checkerboard" images:



Camera Calibration

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Since all points lie in a plane, their component is 0 in world coordinates: $z_w = 0$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ \cancel{z_w} \\ 1 \end{bmatrix}$$

Thus, we can delete the 3rd column of the Extrinsic parameter matrix

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

$$\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3]$$

Camera Calibration

$$\mathbf{h}_1 = K\mathbf{r}_1, \mathbf{h}_2 = K\mathbf{r}_2 \Rightarrow \mathbf{r}_1 = K^{-1}\mathbf{h}_1, \mathbf{r}_2 = K^{-1}\mathbf{h}_2$$

$$\mathbf{r}_1^T \mathbf{r}_1 = \mathbf{r}_2^T \mathbf{r}_2 = 1, \mathbf{r}_1^T \mathbf{r}_2 = 0$$

$$\begin{cases} \mathbf{h}_1^T K^{-T} K^{-1} \mathbf{h}_1 - \mathbf{h}_2^T K^{-T} K^{-1} \mathbf{h}_2 = 0 \\ \mathbf{h}_1^T K^{-T} K^{-1} \mathbf{h}_2 = 0 \end{cases}$$

$B = K^{-T} K^{-1}$ is symmetric and positive definite

K can be calculated from B using Cholesky factorization

Each plane gives us two equations

Since B has 6 degrees of freedom, we need at least 3 different views of a plane

We need at least 4 points per plane

Camera Calibration

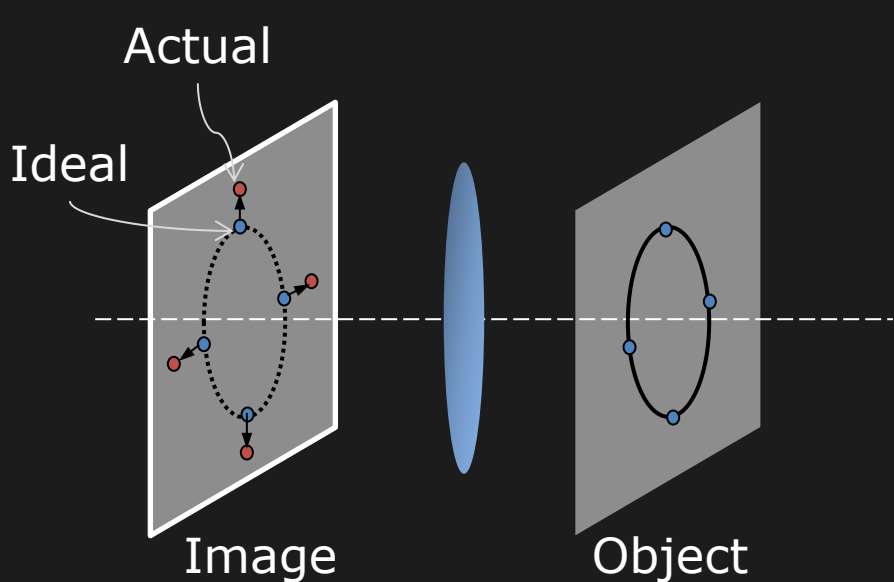
So what's the procedure to calibrate a camera?

Then, we:

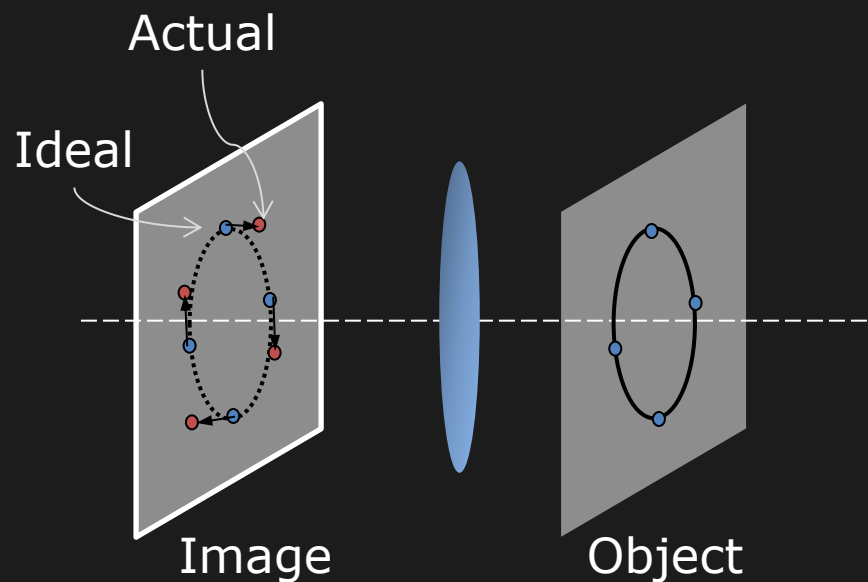
- Extract corners in each checkerboard
- Associate each corner to a 3D location ON the checkerboard
 - Collect across different views
- Solve for camera intrinsics: focal length (in pixels), principal point (in pixels), distortion parameters

Other Intrinsic Parameters: Distortion

Pinholes do not exhibit image distortions. Lenses do.



Radial distortion



Tangential distortion

The mathematical model of the camera will need to incorporate the distortion coefficients.

Distortion Parameters

How do we use the distortion parameters? Given radial distortion coefficients k_1, k_2, k_3 and tangential distortion coefficients p_1 and p_2 :

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + t$$

$$x' = \frac{x}{z} \quad y' = \frac{y}{z}$$

$$x'' = x'(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 x' y' + p_2 (r^2 + 2x'^2)$$

$$y'' = y'(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_2 x' y' + p_1 (r^2 + 2y'^2)$$

where:

$$r^2 = x'^2 + y'^2$$

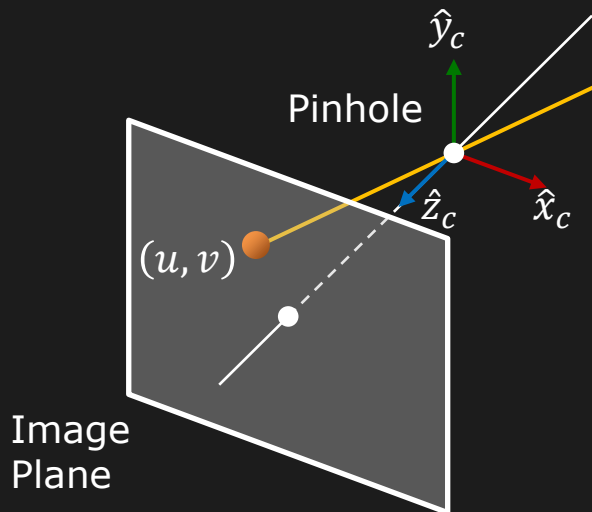
finally:

$$u = f_x x'' + c_x$$

$$v = f_y y'' + c_y$$

Backward Projection: From 2D to 3D

Projection of an image point back into the scene results in an outgoing ray.



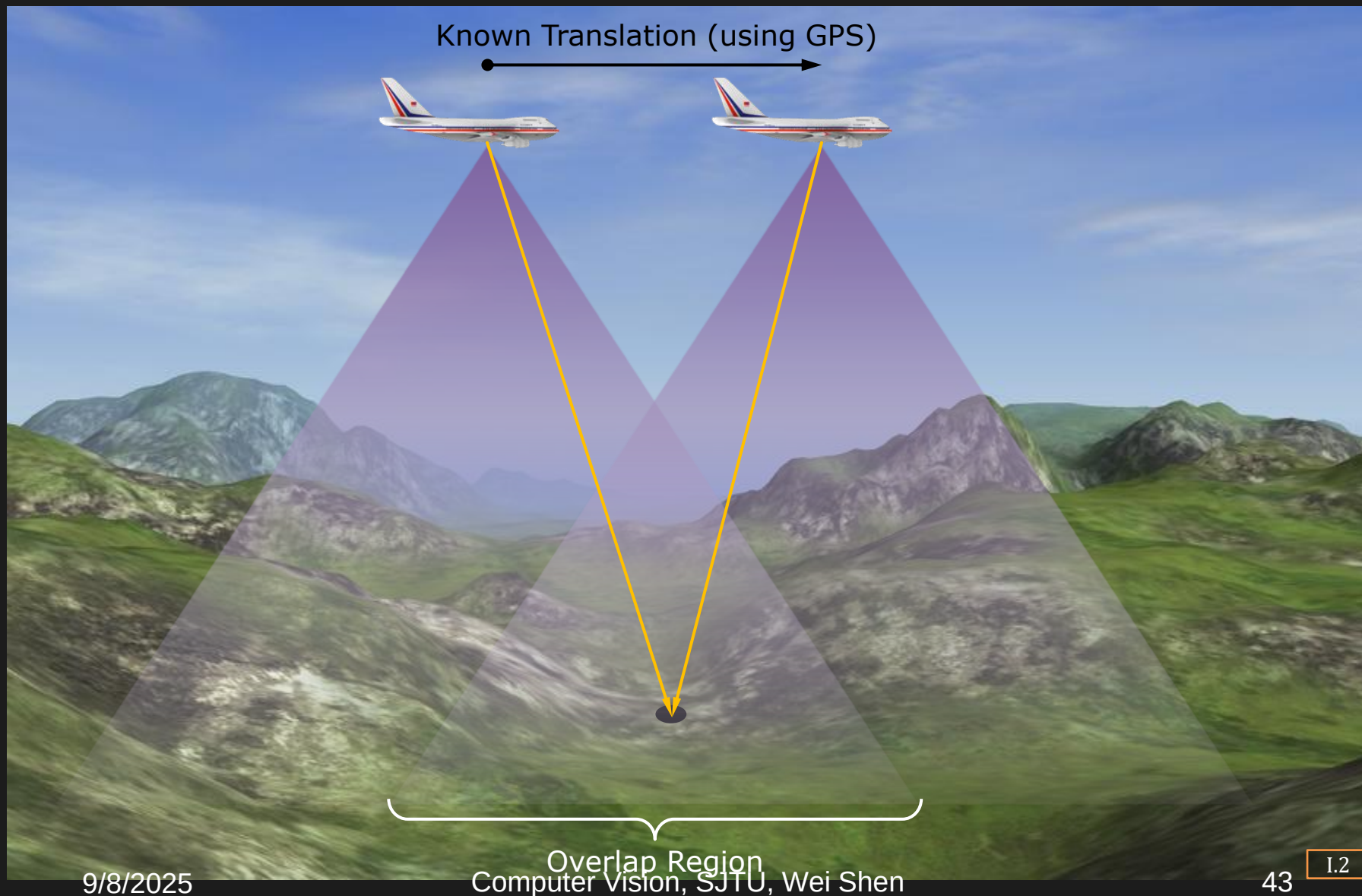
Given a calibrated camera (known intrinsics), what is the direction of the ray?

Pixel location:
$$\begin{aligned} u &= m_x x_i + o_x \\ v &= m_y y_i + o_y \end{aligned}$$

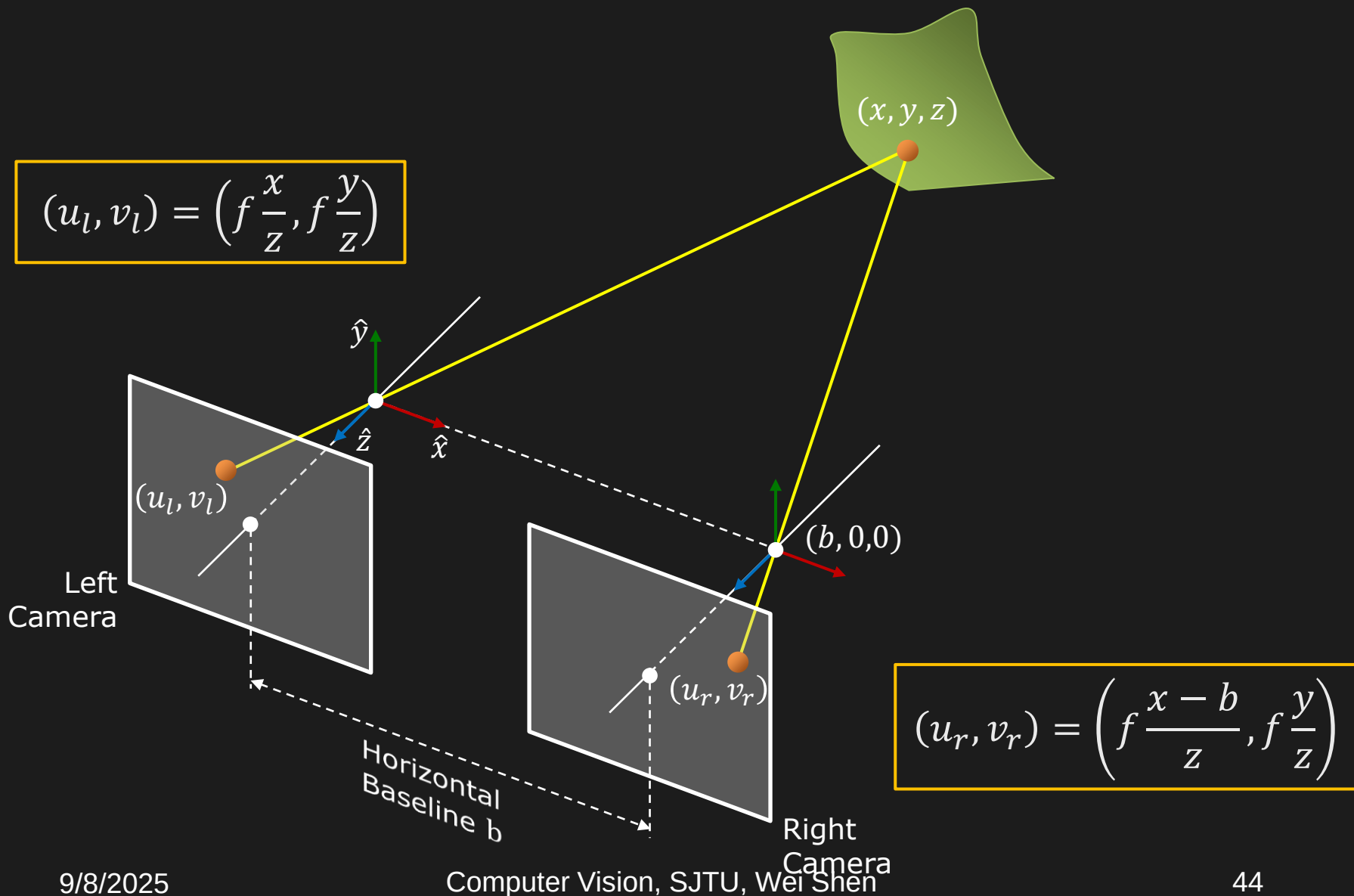
Image point: $\mathbf{x}_i = (x_i, y_i, f)$

Direction of ray:
$$\frac{-\mathbf{x}_i}{\|\mathbf{x}_i\|}$$

Aerial Photogrammetry



Simple Stereo



Simple Stereo: Depth and Disparity

For two pixels corresponding to the same scene point in the left and right images:

$$(u_l, v_l) = \left(f \frac{x}{z}, f \frac{y}{z}\right) \quad (u_r, v_r) = \left(f \frac{x - b}{z}, f \frac{y}{z}\right)$$

Unknown:

Scene point position (x, y, z)

Known:

Pixel positions (u_l, v_l) and (u_r, v_r)

Baseline b from construction (or other calibration methods)

Focal length f from calibration

Simple Stereo: Depth and Disparity

For two pixels corresponding to the same scene point in the left and right images:

$$(u_l, v_l) = \left(f \frac{x}{z}, f \frac{y}{z}\right) \quad (u_r, v_r) = \left(f \frac{x - b}{z}, f \frac{y}{z}\right)$$

Solve for z :

$$\boxed{z = \frac{bf}{(u_l - u_r)}} \longrightarrow x = \frac{zu_l}{f}, \quad y = \frac{zv_l}{f}$$

where $(u_l - u_r)$ is called the **Disparity**.

Given baseline b and focal length f , the scene depth for each pixel can be found by determining the disparity.

Depth is inversely proportional to Disparity.

Simple Stereo: Depth and Disparity

For two pixels corresponding to the same scene point in the left and right images:

$$(u_l, v_l) = \left(f \frac{x}{z}, f \frac{y}{z}\right) \quad (u_r, v_r) = \left(f \frac{x - b}{z}, f \frac{y}{z}\right)$$

Solving for (x, y, z) :

$$x = \frac{bu_l}{(u_l - u_r)} \quad y = \frac{b(v_l + v_r)}{2(u_l - u_r)} \quad z = \frac{bf}{(u_l - u_r)}$$

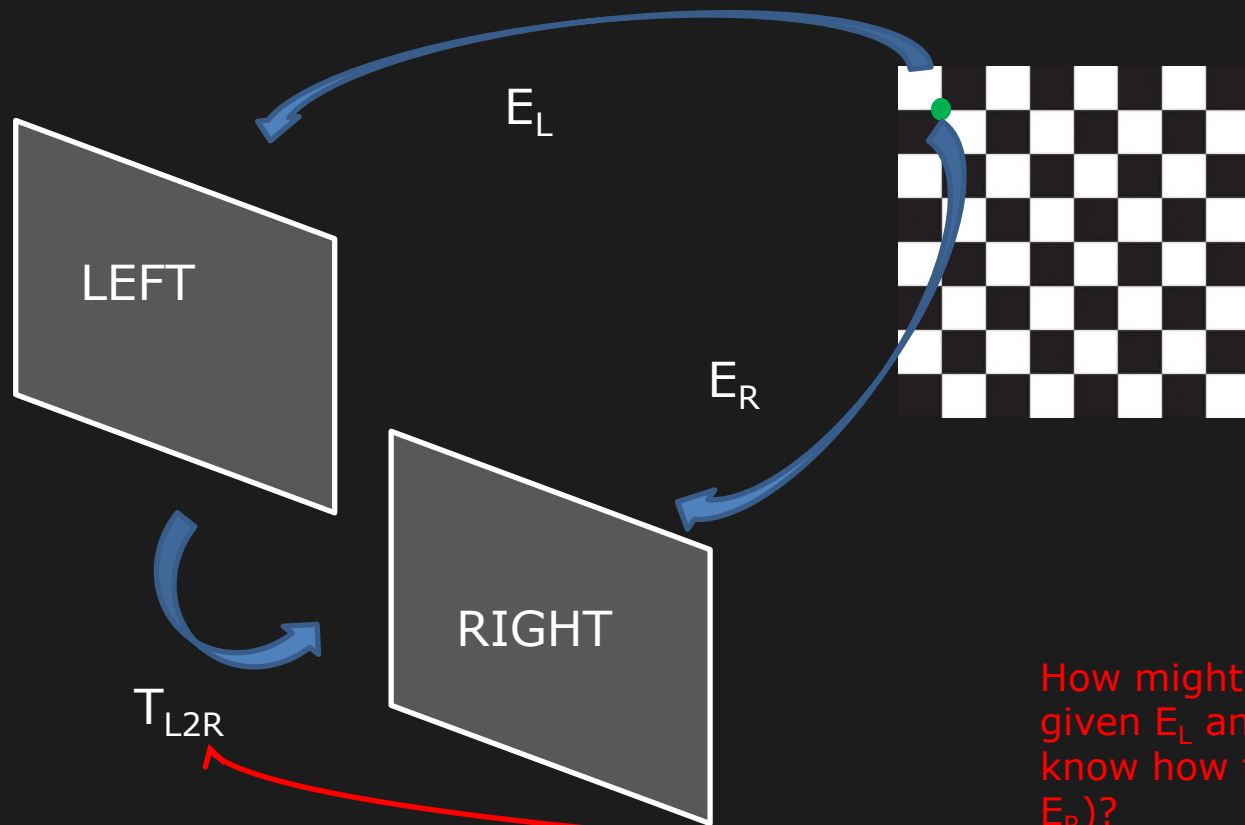
where $(u_l - u_r)$ is called the **Disparity**.

Given baseline b and focal length f , the scene depth for each pixel can be found by determining the disparity.

Depth is inversely proportional to Disparity.

More General Stereo

Stereo cameras differ from each other by a rigid body transform: a 3x3 rotation matrix and a 3x1 translation vector (e.g., the stereo extrinsics)



How might we solve for T_{L2R} given E_L and E_R (since we know how to easily get E_L and E_R)?

Finding Correspondences

Goal: Find the disparity between left and right stereo pairs.



Left Image



Right Image



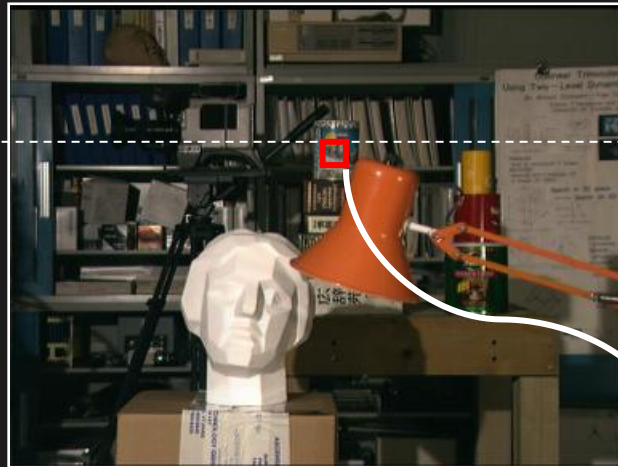
Disparity

From perspective projection: $v_l = v_r = f \frac{y}{z}$

Corresponding scene points lie on the same horizontal scan line.

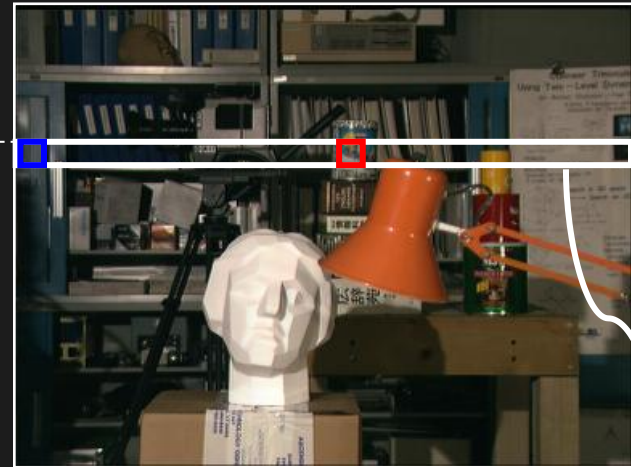
Window based Methods

Determine Disparity using **Template Matching**



Template Window T

Left Camera Image E_l



Search Scan Line L

Right Camera Image E_r

$$\text{Disparity, } d = u_l - u_r$$

Similarity Metrics for Template Matching

Find pixel $(k, l) \in L$ with Minimum **Sum of Absolute Differences**:

$$SAD(k, l) = \sum_{(i, j) \in T} |E_l(i, j) - E_r(i + k, j + l)|$$

Find pixel $(k, l) \in L$ with Minimum **Sum of Squared Differences**:

$$SSD(k, l) = \sum_{(i, j) \in T} |E_l(i, j) - E_r(i + k, j + l)|^2$$

Find pixel $(k, l) \in L$ with Minimum **Normalized Cross-Correlation**:

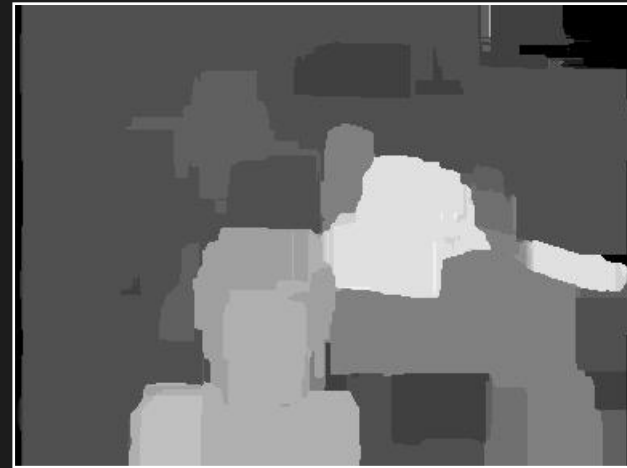
$$NCC(k, l) = \frac{\sum_{(i, j) \in T} |E_l(i, j) - E_r(i + k, j + l)|^2}{\sqrt{\sum_{(i, j) \in T} |E_l(i, j)|^2 \sum_{(i, j) \in T} |E_r(i + k, j + l)|^2}}$$

Issues with Window Based Methods

How large should the window be?



Window size = 5 pixels



Window size = 30 pixels

Adaptive Window Method Solution: For each point, match using windows of multiple sizes and use the disparity that is a result of the best similarity measure.

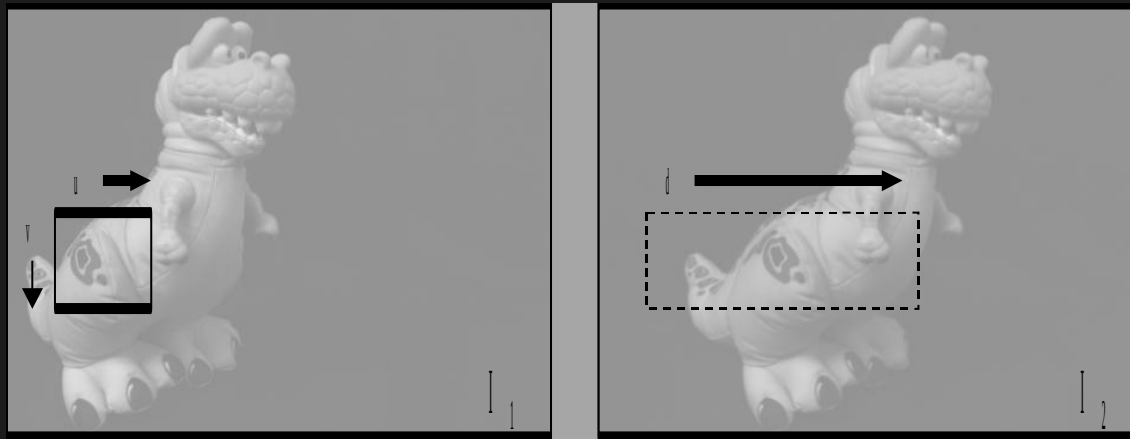
Implementing Region Based Matching

- **Deciding the Range:** The first step in correspondence search is to compute the range of disparities to search
 - Horopter
- Assume a non-verged (rectified) system. Therefore, we have
 - $d = f b / z$
 - given a range $z_{\min}$ to $z_{\max}$, we calculate
 - $d_{\min} = f b / z_{\max}$
 - $d_{\max} = f b / z_{\min}$
- Thus, for each point u_l in the left image, we will search points $u_l + d_{\min}$ to $u_l + d_{\max}$ in the right.
- Note we can turn this around and start at a point u_r and search from $u_r - d_{\max}$ to $u_r - d_{\min}$

Region Based Approach Algorithm

- For each pixel (i,j) of the left image and offset o in disparity range
 - Compute $d(i,j,o) = \sum_{k,l} \psi(I_l(i+k,j+l), I_r(i+k,j+l+o))$
 - The disparity at (i,j) is the value o that minimizes d
- The result of performing this search over every pixel is the *disparity map*.

Correspondence Search Algorithm (simple version for CC)

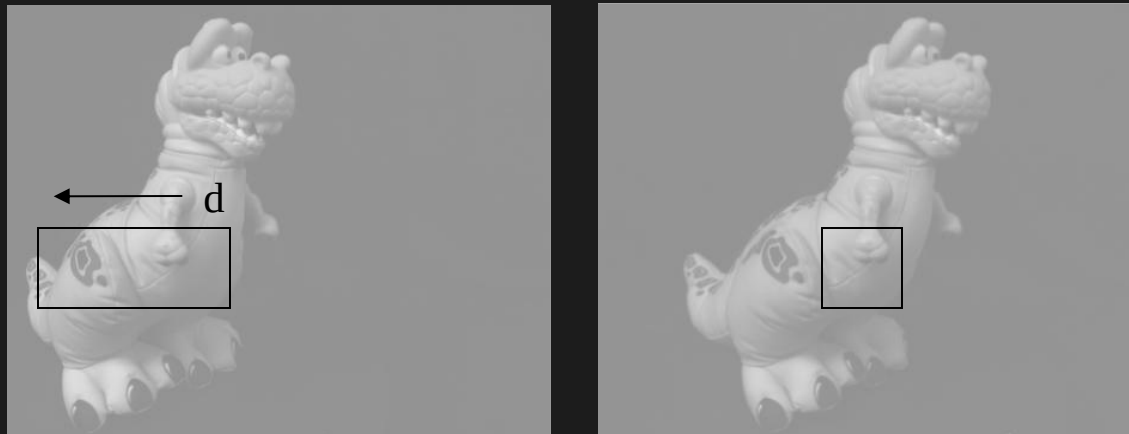


```
For i = 1:nrows
  for j=1:ncols
    best(i,j) = -1
    for k = mindisparity:maxdisparity
      c = CC(I1(i,j),I2(i,j+k),winsize)
      if (c > best(i,j))
        best(i,j) = c
        disparities(i,j) = k
      end
    end
  end
end
```

$O(nrows * ncols * disparities * winx * winy)$

An Additional Twist

- Note that searching from left to right *is not the same* as searching from right to left.
- As a result, we can obtain a somewhat independent disparity map by flipping the images around.
- The results should be the same map up to sign.
- LRCheck: $\text{disp}_l(i,j) = - \text{disp}_r(i,j + \text{disp}_l(i,j))$



Stereo Matching Results



Left Image



Right Image



Ground Truth



SSD (Window size=21)



SSD - Adaptive Window



State of the Art

Challenges of Correspondence

- Photometric issues
 - Specularities
 - Strongly non-Lambertian BRDF's
- Surface structure
 - Lack of texture
 - Repeating texture within horizon bracket
- Geometric ambiguities
 - As surfaces turn away, difficult to get accurate reconstruction (affine approximation can help)
 - At the occluding contour, likelihood of good match but incorrect reconstruction
- Consistency
 - Scanline Consistency
 - Global Optimization

Global Optimization Methods

- Assume that the surfaces are smooth
- Can pose the problem of finding the corresponding points as an energy (or cost) minimization:

$$E([d_1 \dots d_n]) = \sum_i M(I_L(x), I_R(x+d)) + \sum_{j \in N(x)} R(d_i, d_j)$$

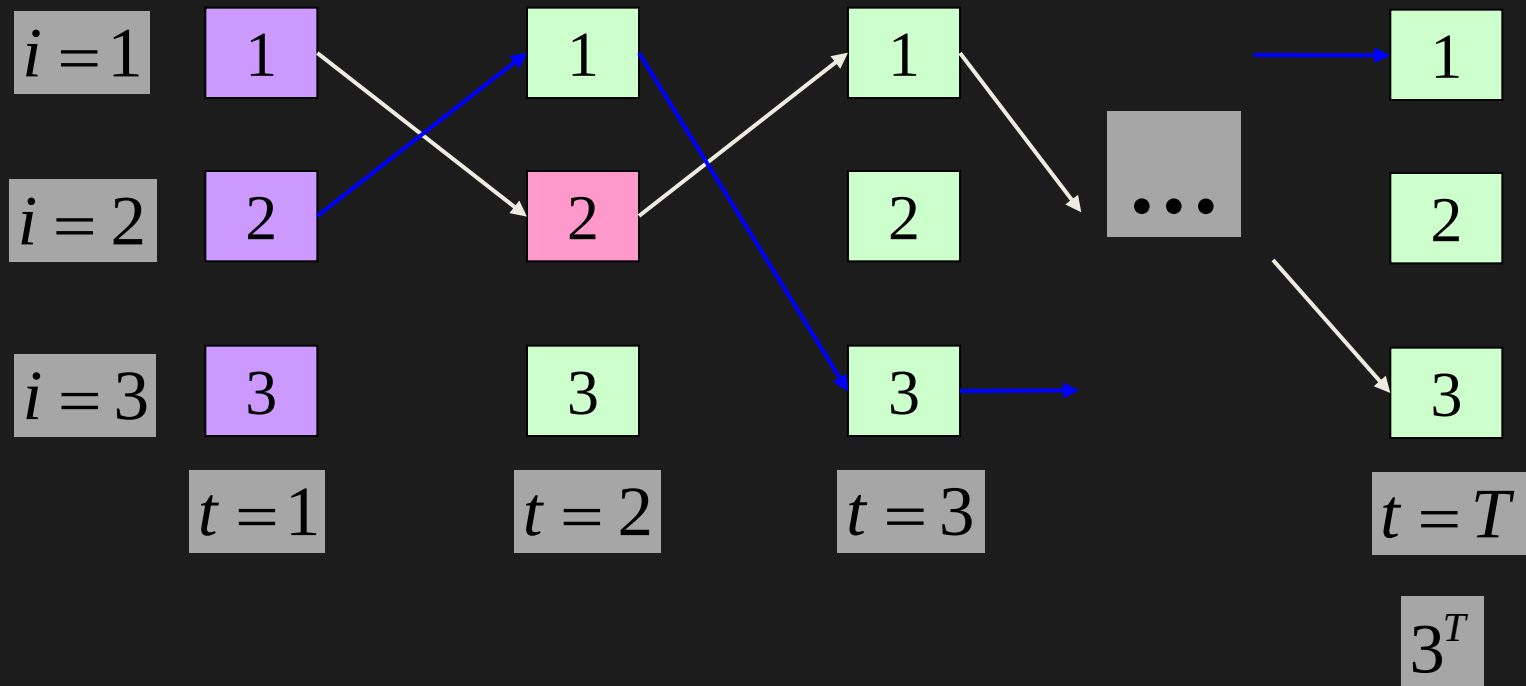
Pixel matching cost

neighboring pixels have similar disparities

- However, this is NP-hard
- Instead use approximations

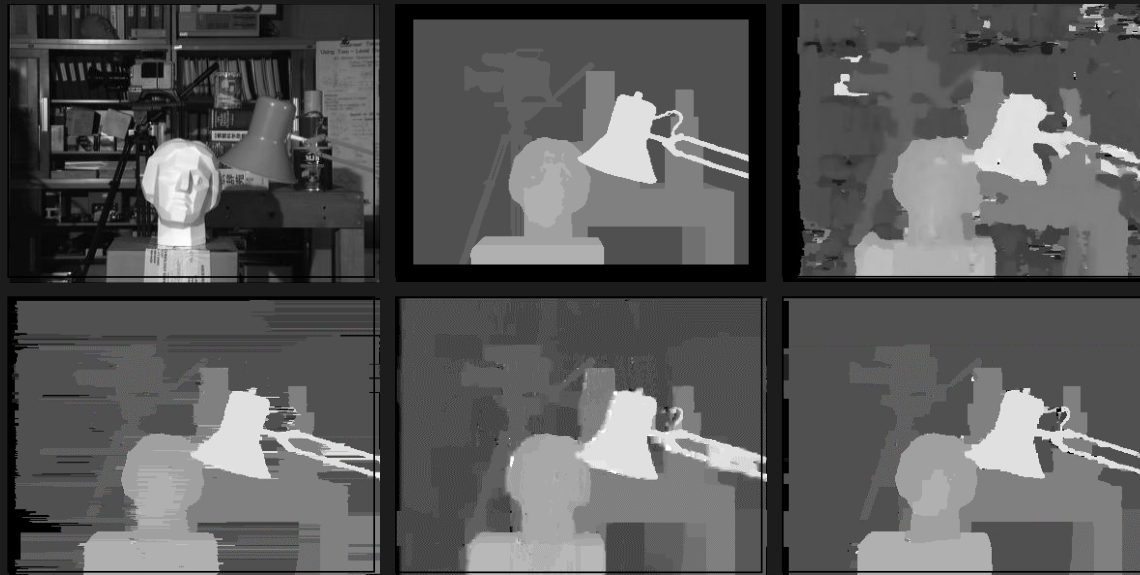
Dynamic Programming

- Efficient algorithm for solving sequential decision (optimal path) problems – a form of **global optimization**.



Local vs. Global Matching

Comparative results on images from the University of Tsukuba, provided by Scharstein and Szeliski [69]. Left to right: left stereo image, ground truth, Muhlmann et al.'s area correlation algorithm [57], dynamic programming (similar to Intille and Bobick [36]), Roy and Cox's maximum flow [65] and Komolgorov and Zabih's graph cuts [45].



Code Packages for Calibration

Two Most Popular

OpenCV (C++, Python):

<http://opencv.willowgarage.com/wiki/>

CalTech Camera Calibration Toolbox (MATLAB):

http://www.vision.caltech.edu/bouguetj/calib_doc/

Appendix: RQ Matrix Factorization

Definition: Any **Invertible Matrix A** can be uniquely decomposed as a product of **Upper Right Triangle matrix R** and **Orthonormal matrix Q**.

$$A = RQ$$

Note that according to our camera model (projection matrix) notation:

$$\underbrace{\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix}}_{P_{3 \times 3}} = \underbrace{\begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}}_K \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}}_R$$

where K is the upper right triangle matrix and R is the orthonormal matrix.

Appendix B: RQ Matrix Factorization

Step 1:

$$\text{Let } A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix}$$

Step 2:

$$\text{Let } R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_x & s_x \\ 0 & -s_x & c_x \end{bmatrix} \quad \text{where} \quad c_x = \frac{a_{33}}{\sqrt{a_{32}^2 + a_{33}^2}} \quad s_x = \frac{a_{32}}{\sqrt{a_{32}^2 + a_{33}^2}}$$

Set $A = AR_x$ (This makes $a_{32} = 0$)

Appendix: RQ Matrix Factorization

Step 3:

$$\text{Let } R_y = \begin{bmatrix} c_y & 0 & -s_y \\ 0 & 1 & 0 \\ s_y & 0 & c_y \end{bmatrix} \quad \text{where} \quad c_y = \frac{a_{33}}{\sqrt{a_{31}^2 + a_{33}^2}} \quad s_y = \frac{-a_{31}}{\sqrt{a_{31}^2 + a_{33}^2}}$$

Set $A = AR_y$ (This makes $a_{31} = 0$)

Step 4:

$$\text{Let } R_z = \begin{bmatrix} c_z & s_z & 0 \\ -s_z & c_z & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{where} \quad c_z = \frac{a_{22}}{\sqrt{a_{21}^2 + a_{22}^2}} \quad s_z = \frac{a_{21}}{\sqrt{a_{21}^2 + a_{22}^2}}$$

Set $K = AR_z$ (This makes $a_{21} = 0$)

Step 5:

$$R = R_z^T R_y^T R_x^T$$

References: Textbooks

Robot Vision (Chapter 13)

Horn, B. K. P., MIT Press

Computer Vision: A Modern Approach (Chapter 10)

Forsyth, D and Ponce, J., Prentice Hall

Multiple View Geometry (Chapters 8-10)

Hartley, R. and Zisserman, A., Cambridge University Press

Computer Vision: Algorithms and Applications (Chapter 7)

Szeliski, R., Springer

An introduction to 3D Computer Vision (Chapter 3)

Cyganek, B., Siebert, J. P., Wiley Pub